

Bases en programmation Matlab

Quentin Rafhay - Jean-Christophe Toussaint

16 septembre 2016

L'ensemble des exercices, fourni ci-après, est à réaliser en totale autonomie et constitue le socle de base en Matlab que vous devez assimiler.

1 Exercice : multiplication matricielle

Matlab est un langage adapté à l'algèbre linéaire et aux calculs matriciels.

1. En utilisant toutes les fonctionnalités de Matlab, développer une fonction `mult(A,B)` retournant le produit matricielle $R = A B$.
2. Pour aider l'utilisateur, compléter la fonction précédente avec un commentaire en début du fichier `mult.m`, donnant un mode d'emploi sommaire.
3. Pour professionnaliser votre code, insérer un bloc permettant de vérifier la compatibilité des tailles des matrices. Utiliser les fonctions `size` et `error`. Cette dernière permet stopper le déroulement de votre programme.
4. Développer une nouvelle fonction `mult2(A, B)` retournant le produit matricielle $R = A B$, dans laquelle l'opération est faite de manière explicite sous la forme de boucles sur les éléments des matrices.
5. En plaçant les fonctions `tic` et `toc` respectivement, avant et après l'appel d'une fonction, Matlab retourne le temps de calcul CPU. Comparer l'efficacité des différentes implémentations proposées en vous limitant aux matrices carrées. Utiliser la fonction `rand` pour générer des matrices de grande taille.

2 Exercice : manipulation de tableaux

On considère un tableau A unidimensionnel de taille $N = 1000$ voire plus.

1. générer le tableau (ligne) A avec la fonction Matlab `rand`.
2. Développer une fonction `min_max(A)` retournant les valeurs minimale et maximale.
3. Développer une fonction `tri_bulle(A)` permettant de trier le tableau de manière croissante. La méthode la plus simple dite de "tri par bulle" consiste à permuter si nécessaire, deux éléments successifs du tableau et à recommencer dès le début du tableau tant qu'il n'est pas trié.

4. Hoare a inventé en 1962, une méthode de type "diviser pour régner" permettant de trier un tableau en $N \ln N$ opérations. Son algorithme s'écrit :

```

1: function TRI_RAPIDE( $T, g, d$ )      14: while true do
2:   if  $g < d$  then                  15:   repeat
3:      $ip, T \leftarrow \text{PARTITION}(T, g, d)$  16:      $j \leftarrow j - 1$ 
4:      $T \leftarrow \text{TRI\_RAPIDE}(T, g, ip)$  17:     until  $T[j] \leq pivot$ 
5:      $T \leftarrow \text{TRI\_RAPIDE}(T, ip+1, d)$  18:     repeat
6:   end if                          19:      $i \leftarrow i + 1$ 
7:   return  $T$                         20:     until  $T[i] \geq pivot$ 
8: end function                     21:     if  $i < j$  then
9:                                     22:        $T \leftarrow \text{ECHANGER}(T, i, j)$ 
10: function PARTITION( $T, g, d$ )       23:     else
11:    $pivot \leftarrow T[g]$            24:       return  $j, T$ 
12:    $i \leftarrow g - 1$                25:     end if
13:    $j \leftarrow d + 1$                26:   end while
                                     27: end function

```

5. La boucle conditionnelle `repeat ... until (condition)` n'existe pas en Matlab, proposez une solution de remplacement. N'oubliez pas de la tester à part.
6. Implémenter l'algorithme suivant sous la forme d'une fonction Matlab. Son appel `A=tri_rapide(A, 1, length(A))` retourne le tableau trié.
7. Faites un programme appelant la fonction `tri_rapide` permettant de trier des tableaux de taille 2^n où $n \in [1, 20]$. En utilisant les fonctions `tic` et `toc`, stocker le temps CPU dans un tableau et finalement tracer l'évolution du temps de calcul en fonction de la taille.
8. Vérifier qu'il varie en $N \ln N$ dans la limite des grandes valeurs de N .

3 Exercice : nombres premiers

On désire connaître la liste L_p des nombres premiers inférieurs à N entier donné.

1. Développer une fonction Matlab retournant cette liste.

Indication : on peut construire la liste de manière dynamique sans allocation préalable. L'initialisation de la liste vide se fait par l'instruction `Lp=[ ]` et l'insertion à droite d'un élément e dans L_p par `Lp = [Lp, e]`.

4 Méthode de la puissance itérée

4.1 Notations

On considère une matrice réelle symétrique ou hermitique A de dimensions $n \times n$. Elle vérifie par conséquent la propriété $A = (A^t)^*$ où $*$ est l'opérateur de conjugaison complexe que l'on applique à chaque élément de A^t . On sait aussi que dans ce cas, A est toujours diagonalisable dans $\mathbb{C}$.

On définit le produit scalaire de deux vecteurs $|u\rangle = (x_1, x_2, \dots, x_n)^t$ et $|v\rangle = (y_1, y_2, \dots, y_n)^t$ par :

$$\langle v|u\rangle = \sum_{i=1}^n x_i y_i^* \quad (1)$$

Supposons pour simplifier que le spectre de ses valeurs propres est non dégénéré. Autrement dit, à chaque valeur propre λ_i correspond un s.e.v. de dimension 1, décrit par un vecteur propre $|u_i\rangle$. La base formée par l'ensemble des vecteurs propres est orthogonale et peut être normée.

On peut décomposer A comme une somme de projecteurs :

$$A = \sum_{i=1}^n \lambda_i |u_i\rangle \langle u_i| \quad (2)$$

4.2 Recherche de la plus grande valeur propre en module

On désire dans un premier temps extraire la plus grande valeur propre en module que nous nommons λ_1 et définir le vecteur propre associé $|u_1\rangle$ en utilisant une méthode itérative.

Décrivons cet algorithme. On choisit d'abord un vecteur aléatoire $|x_0\rangle$ de dimension n sur lequel on applique p fois la matrice A .

A la première itération, on a :

$$|x_1\rangle = A|x_0\rangle = \sum_{i=1}^n \lambda_i |u_i\rangle \langle u_i|x_0\rangle \quad (3)$$

A l'itération p , on obtient :

$$|x_p\rangle = A^p|x_0\rangle = \sum_{i=1}^n \lambda_i^p |u_i\rangle \langle u_i|x_0\rangle \quad (4)$$

que l'on peut développer suivant la forme suivante :

$$|x_p\rangle = A^p|x_0\rangle = \lambda_1^p \left(\langle u_1|x_0\rangle |u_1\rangle + \left(\frac{\lambda_2}{\lambda_1}\right)^p \langle u_2|x_0\rangle |u_2\rangle + \dots + \left(\frac{\lambda_n}{\lambda_1}\right)^p \langle u_n|x_0\rangle |u_n\rangle \right) \quad (5)$$

En remarquant que $|\frac{\lambda_i}{\lambda_1}| \leq 1$ et dans la limite des grandes valeurs de p , on observe que :

$$\lim_{p \rightarrow \infty} |x_p\rangle \propto \langle u_1|x_0\rangle |u_1\rangle \quad (6)$$

Le vecteur $|x_p\rangle$ tend à s'aligner vers le vecteur propre $|u_1\rangle$.

4.3 Implémentation de la méthode en Matlab/Octave

1. On donne ci-dessous un algorithme simplifié mettant en oeuvre la méthode de la puissance itérée.

```

1: function PUISSANCE( $A, x$ )           8:       $iter \leftarrow iter + 1$ 
2:    $iter \leftarrow 0$                  9:      until  $NORM(x - x0) < \epsilon$ 
3:    $\epsilon \leftarrow 10^{-6}$           10:
4:   repeat                           11:    $\lambda \leftarrow x^t A x$ 
5:      $x0 \leftarrow x$                  12:   return  $\lambda, x$ 
6:      $v \leftarrow A x$                 13: end function
7:      $x \leftarrow v / NORM(v)$ 

```

2. Implémenter l'algorithme précédent sous la forme d'une fonction Matlab `[lb, u] = puissance(A, x)` où A est une matrice de dimensions $n \times n$ et un vecteur colonne x non nul, de dimension n . Elle retourne deux variables `lb` et `u` qui correspondent à la valeur propre λ_1 et au vecteur propre $|u_1\rangle$.
3. Compléter le fichier `puissance.m` avec un en-tête de commentaires résumant l'usage de la fonction.
4. Tester son comportement depuis la fenêtre de commande, en prenant $A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 1 & 2 \\ 3 & 2 & 1 \end{pmatrix}$.
5. Vérifier que les résultats sont cohérents avec ceux donnés par la fonction interne à Matlab `eig(A)` et `[V,D] = eig(A)`. Commenter.
6. Modifier la fonction pour récupérer le nombre d'itérations. Quel est l'intérêt de le faire ?
7. Exécuter de nouveau cette fonction en prenant comme matrice $-A$. Commenter.
8. Que se passe-t-il si la plus grande valeur propre en module est dégénérée ? La méthode fonctionne-t-elle encore dans ce cas ?
9. On peut générer une matrice aléatoire de grande taille avec la fonction `rand(N, N)` dont la valeur de chaque élément est dans $[0, 1[$. Comment la rendre symétrique ?
10. Est-il raisonnable d'utiliser cet algorithme dans le cas d'une matrice non symétrique ?
11. Est-il raisonnable d'utiliser cet algorithme dans le cas d'une matrice hermitique ?
12. Modifier la fonction `[l, u] = puissance(A, x)` en imposant un nombre maximal d'itérations. Quelles en sont les conséquences immédiates ?
13. Proposez une approche, même si elle n'est pas efficace, pour extraire la plus petite valeur propre en module d'une matrice.
14. Montrer qu'en modifiant la ligne 6 de l'algorithme précédent, la plus petite valeur propre λ_{min} et son vecteur propre associé peuvent être calculés à condition toutefois que $\lambda_{min} > 0$.